

part H. Classes

The world of JavaScript during TypeScript’s creation and release in the early 2010s was quite different from today.

Features such as arrow functions and `let / const` variables that would later be standardized in ES2015 were still distant hopes on the horizon. Babel was a few years away from its first commit; its predecessor tools such as Traceur that converted newer JavaScript syntax to old hadn’t achieved full mainstream adoption.

TypeScript’s early marketing and feature set were tailored to that world. In addition to its type checking, its transpiler was emphasized—with classes as a frequent example. Nowadays TypeScript’s class support is just one feature among many to support all JavaScript language features. TypeScript neither encourages nor discourages class use or any other popular JavaScript pattern.

Class Methods

TypeScript generally understands methods the same way it understands standalone functions. Parameter types default to `any` unless given a type or default value; calling the method requires an acceptable number of arguments; return types can generally be inferred if the function is not recursive.

This code snippet defines a `Greeter` class with a `greet` class method that takes in a single required parameter of type `string`:

```
class Greeter {
  greet(name: string) {
    console.log(`${name}, do your stuff!`);
  }
}

new Greeter().greet("Miss Frizzle"); // Ok

new Greeter().greet();
```

```
// ~~~~~  
// Error: Expected 1 arguments, but got 0.
```

Class constructors are treated like typical class methods with regards to their parameters. TypeScript will perform type checking to make sure a correct number of arguments with correct types are provided to method calls.

This `Greeted` constructor also expects its `message: string` parameter to be provided:

```
class Greeted {  
    constructor(message: string) {  
        console.log(`As I always say: ${message}!`);  
    }  
}  
  
new Greeted("take chances, make mistakes, get messy");  
  
new Greeted();  
// Error: Expected 1 arguments, but got 0.
```

I'll cover constructors in the context of subclasses later in this part.

Class Properties

To read from or write to a property on a class in TypeScript, it must be explicitly declared in the class. Class properties are declared using the same syntax as interfaces: their name followed optionally by a type annotation.

TypeScript will not attempt to deduce what members may exist on a class from their assignments in a constructor.

In this example, `destination` is allowed to be assigned to and accessed on instances of the `FieldTrip` class because it is explicitly declared as a `string`. The `this.nonexistent` assignment in the constructor is not allowed because the class does not declare a `nonexistent` property:

```
class FieldTrip {  
    destination: string;  
  
    constructor(destination: string) {
```

```

    this.destination = destination; // Ok
    console.log(`We're going to ${this.destination}!`);

    this.nonexistent = destination;
    //      ~~~~~
    // Error: Property 'nonexistent' does not exist on type 'FieldTrip'.
  }
}

```

Explicitly declaring class properties allows TypeScript to quickly understand what is or is not allowed to exist on instances of classes. Later, when class instances are in use, TypeScript uses that understanding to give a type error if code attempts to access a member of a class instance not known to exist, such as with this continuation's

```
trip.nonexistent:
```

```

const trip = new FieldTrip("planetarium");

trip.destination; // Ok

trip.nonexistent;
//      ~~~~~
// Error: Property 'nonexistent' does not exist on type 'FieldTrip'.

```

Function Properties

Let's recap some JavaScript method scoping and syntax fundamentals for a bit, as they can be surprising if you're not accustomed to them. JavaScript contains two syntaxes for declaring a member on a class to be a callable function: *method* and *property*.

I've already shown the method approach of putting parentheses after the member name, like `myFunction() {}`. The method approach assigns a function to the class prototype, so all class instances use the same function definition.

This `WithMethod` class declares a `myMethod` method that all instances are able to refer to:

```

class WithMethod {
  myMethod() {}
}

```

```
new WithMethod().myMethod === new WithMethod().myMethod; // true
```

The other syntax is to declare a property whose value happens to be a function. This creates a new function per instance of the class, which can be useful with `() =>` arrow functions whose `this` scope should always point to the class instance (at the time and memory cost of creating a new function per class instance).

This `WithProperty` class contains a single property of name `myProperty` and type `() => void` that will be re-created for each class instance:

```
class WithProperty {  
    myProperty = () => {}  
}  
  
new WithProperty().myProperty === new WithProperty().myProperty; // false
```

Function properties can be given parameters and return types using the same syntax as class methods and stand-alone functions. After all, they're a value assigned to a class member and the value happens to be a function.

This `WithPropertyParameters` class has a `takesParameters` property of type `(input: boolean) => number`:

```
class WithPropertyParameters {  
    takesParameters = (input: boolean) => input ? "Yes" : "No";  
}  
  
const instance = new WithPropertyParameters();  
  
instance.takesParameters(true); // Ok  
  
instance.takesParameters(123);  
//  
// Error: Argument of type 'number' is not  
// assignable to parameter of type 'boolean'.
```

Initialization Checking

With strict compiler settings enabled, TypeScript will check that each property declared whose type does not include `undefined` is assigned a value in the constructor. This strict initialization checking is useful because it prevents code from accidentally forgetting to assign a value to a class property.

The following `WithValue` class does not assign a value to its `unused` property, which TypeScript recognizes as a type error:

```
class WithValue {
  immediate = 0; // Ok
  later: number; // Ok (set in the constructor)
  maybeUndefined: number | undefined; // Ok (allowed to be undefined)

  unused: number;
  // Error: Property 'unused' has no initializer
  // and is not definitely assigned in the constructor.

  constructor() {
    this.later = 1;
  }
}
```

Without strict initialization checking, a class instance could be allowed to access a value that might be `undefined` even though the type system says it can't be.

This example would compile happily if strict initialization checking didn't happen, but the resultant JavaScript would crash at runtime:

```
class MissingInitializer {
  property: string;
}

new MissingInitializer().property.length;
// TypeError: Cannot read property 'length' of undefined
```

The billion-dollar mistake strikes again!

Definitely assigned properties

Although strict initialization checking is useful most of the time, you may come across some cases where a class property is intentionally able to be unassigned after the class constructor. If you are absolutely sure a property should not have strict initialization checking applied to it, you can add a `!` after its name to disable the check. Doing so asserts to TypeScript that the property will be assigned a value other than `undefined` before its first usage.

This `ActivitiesQueue` class is meant to be re-initialized any number of times separately from its constructor, so its `pending` property must be asserted with a `!`:

```
class ActivitiesQueue {
  pending!: string[]; // Ok

  initialize(pending: string[]) {
    this.pending = pending;
  }

  next() {
    return this.pending.pop();
  }
}

const activities = new ActivitiesQueue();

activities.initialize(['eat', 'sleep', 'learn'])
activities.next();
```

WARNING

Needing to disable strict initialization checking on a class property is often a sign of code being set up in a way that doesn't lend itself well to type checking. Instead of adding a `!` assertion and reducing type safety for the property, consider refactoring the class to no longer need the assertion.

Optional Properties

Much like interfaces, classes in TypeScript may declare a property as optional by adding a `?` after its declaration name. Optional properties behave roughly the same as properties whose types happen to be a union that includes `|`

`undefined`. Strict initialization checking won't mind if they're not explicitly set in their constructor.

This `MissingInitializer` class marks its `property` as optional, so it's allowed to not be assigned in the class constructor regardless of strict property initialization checking:

```
class MissingInitializer {  
    property?: string;  
}  
  
new MissingInitializer().property?.length; // Ok  
  
new MissingInitializer().property.length;  
// Error: Object is possibly 'undefined'.
```

Read-Only Properties

Again much like interfaces, classes in TypeScript may declare a property as read-only by adding the `readonly` keyword before its declaration name. The `readonly` keyword exists purely within the type system and is removed when compiling to JavaScript.

Properties declared as `readonly` may only be assigned initial values where they are declared or in a constructor. Any other location—including methods on the class itself—may only read from the properties, not write to them.

In this example, the `text` property on the `Quote` class is given a value in the constructor, but the other uses cause type errors:

External users of your code, such as consumers of any npm packages you published, might not respect `readonly` modifiers—especially if they're writing JavaScript and don't have type checking. If you need true read-only protection, consider using `#` private fields and/or `get()` function properties.

```
class Quote {  
    readonly text: string;  
  
    constructor(text: string) {  
        this.text = text;  
    }  
}
```



```

    emphasize() {
        this.text += "!";
        // ~~~~
        // Error: Cannot assign to 'text' because it is a read-only property.
    }
}

const quote = new Quote(
    "There is a brilliant child locked inside every student."
);

quote.text = "Ha!";
// Error: Cannot assign to 'text' because it is a read-only property.

```

Properties declared as `readonly` with an initial value of a primitive have a slight quirk compared to other properties: they are inferred to be their value's narrowed *literal* type if possible, rather than the wider *primitive*. TypeScript feels comfortable with a more aggressive initial type narrowing because it knows the value won't be changed later; it is similar to `const` variables taking on narrower types than `let` variables.

In this example, the class properties are both initially declared as a string literal, so in order to widen one of them to `string`, a type annotation is needed:

```

class RandomQuote {
    readonly explicit: string = "Home is the nicest word there is.";
    readonly implicit = "Home is the nicest word there is.";

    constructor() {
        if (Math.random() > 0.5) {
            this.explicit = "We start learning the minute we're born." // Ok;

            this.implicit = "We start learning the minute we're born.";
            // Error: Type '"We start learning the minute we're born."' is
            // not assignable to type '"Home is the nicest word there is."'.
        }
    }
}

const quote = new RandomQuote();

```



```
quote.explicit; // Type: string
quote.implicit; // Type: "Home is the nicest word there is."
```

Widening a property’s type explicitly is not necessary very often. Still, it can sometimes be useful in the case of conditional logic in constructors like the one in `RandomQuote`.

Classes as Types

Classes are relatively unique in the type system in that a class declaration creates both a runtime value—the class itself—as well as a type that can be used in type annotations.

The name of this `Teacher` class is used to annotate a `teacher` variable, telling TypeScript that it should be assigned only values that are assignable to the `Teacher` class—such as instances of the `Teacher` class itself:

```
class Teacher {
  sayHello() {
    console.log("Take chances, make mistakes, get messy!");
  }
}

let teacher: Teacher;

teacher = new Teacher(); // Ok

teacher = "Wahoo!";
// Error: Type 'string' is not assignable to type 'Teacher'.
```

Interestingly, TypeScript will consider any object type that happens to include all the same members of a class to be assignable to the class. This is because TypeScript’s structural typing cares only about the shape of objects, not how they’re declared.

Here, `withSchoolBus` takes in a parameter of type `SchoolBus`. That can be satisfied by any object that happens to have a `getAbilities` property of type `() => string[]`, such as an instance of the `SchoolBus` class:

```
class SchoolBus {
  getAbilities() {
    return ["magic", "shapeshifting"];
  }
}
```

```

    }
}

function withSchoolBus(bus: SchoolBus) {
    console.log(bus.getAbilities());
}

withSchoolBus(new SchoolBus()); // Ok

// Ok
withSchoolBus({
    getAbilities: () => ["transmogrification"],
});

withSchoolBus({
    getAbilities: () => 123,
    // ~~~
    // Error: Type 'number' is not assignable to type 'string[]'.
});

```

Classes and Interfaces

Back in , I showed you how interfaces allow TypeScript developers to set up expectations for object shapes in code. TypeScript allows a class to declare its instances as adhering to an interface by adding the `implements` keyword after the class name, followed by the name of an interface. Doing so indicates to TypeScript that instances of the class should be assignable to each of those interfaces. Any mismatches would be called out as type errors by the type checker.

In this example, the `Student` class correctly implements the `Learner` interface by including its property `name` and method `study`, but `Slacker` is missing a `study` and thus results in a type error:

```

interface Learner {
    name: string;
    study(hours: number): void;
}

class Student implements Learner {
    name: string;

    constructor(name: string) {
        this.name = name;
    }
}

```

```

    study(hours: number) {
        for (let i = 0; i < hours; i+= 1) {
            console.log("...studying...");
        }
    }
}

class Slacker implements Learner {
    // ~~~~~
    // Error: Class 'Slacker' incorrectly implements interface 'Learner'.
    // Property 'study' is missing in type 'Slacker'
    // but required in type 'Learner'.
    name = "Rocky";
}

```

Marking a class as implementing an interface doesn't change anything about how the class is used. If the class already happened to match up to the interface, TypeScript's type checker would have allowed its instances to be used in places where an instance of the interface is required anyway. TypeScript won't even infer the types of methods or properties on the class from the interface: if we had added a `study(hours) {}` method to the `Slacker` example, TypeScript would consider the `hours` parameter an implicit `any` unless we gave it a type annotation.

This version of the `Student` class causes implicit `any` type errors because it doesn't provide type annotations on its members:

```

class Student implements Learner {
    name;
    // Error: Member 'name' implicitly has an 'any' type.

    study(hours) {
        // Error: Parameter 'hours' implicitly has an 'any' type.
    }
}

```

Implementing an interface is purely a safety check. It does not copy any interface members onto the class definition for you. Rather, implementing an interface signals your intention to the type checker and surfaces type errors in the class definition, rather than later on where class instances are

used. It's similar in purpose to adding a type annotation to a variable even though it has an initial value.

Implementing Multiple Interfaces

Classes in TypeScript are allowed to be declared as implementing multiple interfaces. The list of implemented interfaces for a class may be any number of interface names with commas in-between.

In this example, both classes are required to have at least a `grades` property to implement `Graded` and a `report` property to implement `Reporter`. The `Empty` class has two type errors for failing to implement either of the interfaces properly:

```
interface Graded {
    grades: number[];
}

interface Reporter {
    report: () => string;
}

class ReportCard implements Graded, Reporter {
    grades: number[];

    constructor(grades: number[]) {
        this.grades = grades;
    }

    report() {
        return this.grades.join(", ");
    }
}

class Empty implements Graded, Reporter { }
// ~~~~~
// Error: Class 'Empty' incorrectly implements interface 'Graded'.
//   Property 'grades' is missing in type 'Empty'
//   but required in type 'Graded'.
// ~~~~~
// Error: Class 'Empty' incorrectly implements interface 'Reporter'.
//   Property 'report' is missing in type 'Empty'
//   but required in type 'Reporter'.
```

In practice, there may be some interfaces whose definitions make it impossible to have a class implement both.

Attempting to declare a class implementing two conflicting interfaces will result in at least one type error on the class.

The following `AgeIsANumber` and `AgeIsNotANumber` interfaces declare very different types for an `age` property.

Neither the `AsNumber` class nor `NotAsNumber` class properly implement both:

```
interface AgeIsANumber {
    age: number;
}

interface AgeIsNotANumber {
    age: () => string;
}

class AsNumber implements AgeIsANumber, AgeIsNotANumber {
    age = 0;
    // ~~~
    // Error: Property 'age' in type 'AsNumber' is not assignable
    // to the same property in base type 'AgeIsNotANumber'.
    //   Type 'number' is not assignable to type '() => string'.
}
```

```
class NotAsNumber implements AgeIsANumber, AgeIsNotANumber {
    age() { return ""; }
    // ~~~
    // Error: Property 'age' in type 'NotAsNumber' is not assignable
    // to the same property in base type 'AgeIsANumber'.
    //   Type '() => string' is not assignable to type 'number'.
}
```

Cases where two interfaces describe very different object shapes generally indicate you shouldn't try to implement them with the same class.

Extending a Class

TypeScript adds type checking onto the JavaScript concept of a class extending, or subclassing, another class. To start, any method or property declared on a base class will be available on the subclass, also known as the derived class.

In this example, `Teacher` declares a `teach` method that may be used by instances of the `StudentTeacher` subclass:

```
class Teacher {
  teach() {
    console.log("The surest test of discipline is its absence.");
  }
}

class StudentTeacher extends Teacher {
  learn() {
    console.log("I cannot afford the luxury of a closed mind.");
  }
}

const teacher = new StudentTeacher();
teacher.teach(); // Ok (defined on base)
teacher.learn(); // Ok (defined on subclass)

teacher.other();
//      ~~~~~
// Error: Property 'other' does not exist on type 'StudentTeacher'.
```

Extension Assignability

Subclasses inherit members from their base class much like derived interfaces extend base interfaces. Instances of subclasses have all the members of their base class and thus may be used wherever an instance of the base is required. If a base class doesn't have all the members a subclass does, then it can't be used when the more specific subclass is required.

Instances of the following `Lesson` class may not be used where instances of its derived `OnlineLesson` are required, but derived instances may be used to satisfy either the base or subclass:

```
class Lesson {
  subject: string;

  constructor(subject: string) {
    this.subject = subject;
  }
}

class OnlineLesson extends Lesson {
```



```

    url: string;

    constructor(subject: string, url: string) {
        super(subject);
        this.url = url;
    }
}

let lesson: Lesson;
lesson = new Lesson("coding"); // Ok
lesson = new OnlineLesson("coding", "oreilly.com"); // Ok

let online: OnlineLesson;
online = new OnlineLesson("coding", "oreilly.com"); // Ok

online = new Lesson("coding");
// Error: Property 'url' is missing in type
// 'Lesson' but required in type 'OnlineLesson'.

```

Per TypeScript's structural typing, if all the members on a subclass already exist on its base class with the same type, then instances of the base class are still allowed to be used in place of the subclass.

In this example, `LabeledPastGrades` only adds an optional property to `PastGrades`, so instances of the base class may be used in place of the subclass:

```

class PastGrades {
    grades: number[] = [];
}

class LabeledPastGrades extends PastGrades {
    label?: string;
}

let subClass: LabeledPastGrades;

subClass = new LabeledPastGrades(); // Ok
subClass = new PastGrades(); // Ok

```

Overridden Constructors

As with vanilla JavaScript, subclasses are not required by TypeScript to define their own constructor. Subclasses without their own constructor implicitly use the constructor from their base class.

In JavaScript, if a subclass does declare its own constructor, then it must call its base class constructor via the `super` keyword. Subclass constructors may declare any parameters regardless of what their base class requires. TypeScript's type checker will make sure that the call to the base class constructor uses the correct parameters.

In this example, `PassingAnnouncer`'s constructor correctly calls the base constructor with a `number` argument, while `FailingAnnouncer` gets a type error for forgetting to make that call:

```
class GradeAnnouncer {
    message: string;

    constructor(grade: number) {
        this.message = grade < 65 ? "Maybe next time..." : "You pass!";
    }
}

class PassingAnnouncer extends GradeAnnouncer {
    constructor() {
        super(100);
    }
}

class FailingAnnouncer extends GradeAnnouncer {
    constructor() { }
    // ~~~~~
    // Error: Constructors for subclasses must contain a 'super' call.
}
```

As per JavaScript rules, the constructor of a subclass must call the base constructor before accessing `this` or `super`. TypeScript will report a type error if it sees a `this` or `super` being accessed before `super()`.

The following `ContinuedGradesTally` class erroneously refers to `this.grades` in its constructor before calling to `super()`:

```
class GradesTally {
    grades: number[] = [];

    addGrades(...grades: number[]) {
        this.grades.push(...grades);
        return this.grades.length;
    }
}
```

```

    }
}

class ContinuedGradesTally extends GradesTally {
  constructor(previousGrades: number[]) {
    this.grades = [...previousGrades];
    // Error: 'super' must be called before accessing
    // 'this' in the constructor of a subclass.

    super();

    console.log("Starting with length", this.grades.length); // Ok
  }
}

```

Overridden Methods

Subclasses may redeclare new methods with the same names as the base class, as long as the method on the subclass is assignable to the method on the base class.

Remember, since subclasses can be used wherever the original class is used, the types of the new methods must be usable in place of the original methods.

In this example, `FailureCounter`'s `countGrades` method is permitted because it has the same first parameter and return type as the base `GradeCounter`'s `countGrades` method. `AnyFailureChecker`'s `countGrades` causes a type error for having the wrong return type:

```

class GradeCounter {
  countGrades(grades: string[], letter: string) {
    return grades.filter(grade => grade === letter).length;
  }
}

class FailureCounter extends GradeCounter {
  countGrades(grades: string[]) {
    return super.countGrades(grades, "F");
  }
}

class AnyFailureChecker extends GradeCounter {
  countGrades(grades: string[]) {
    // Property 'countGrades' in type 'AnyFailureChecker' is not
    // assignable to the same property in base type 'GradeCounter'.
    //   Type '(grades: string[]) => boolean' is not assignable
    //   to type '(grades: string[], letter: string) => number'.
  }
}

```

```

//      Type 'boolean' is not assignable to type 'number'.
return super.countGrades(grades, "F") !== 0;
}
}

const counter: GradeCounter = new AnyFailureChecker();
//      ~~~~~
// Type 'AnyFailureChecker' is not assignable to type 'GradeCounter'.
//   The types returned by 'countGrades(...)' are incompatible between these types.
//      Type 'boolean' is not assignable to type 'number'.(2322)

// Expected type: number
// Actual type: boolean
const count = counter.countGrades(["A", "C", "F"], "C");

```

Overridden Properties

Subclasses may also explicitly redeclare properties of their base class with the same name, as long as the new type is assignable to the type on the base class. As with overridden methods, subclasses must structurally match up with base classes.

Most subclasses that redeclare properties do so either to make those properties a more specific subset of a type union or to make the properties a type that extends from the base class property's type.

In this example, the base class `Assignment` declares its `grade` to be `number | undefined`, while the subclass `GradedAssignment` declares it as a `number` that must always exist:

```

class Assignment {
    grade?: number;
}

class GradedAssignment extends Assignment {
    grade: number;

    constructor(grade: number) {
        super();
        this.grade = grade;
    }
}

```

Expanding the allowed set of values of a property's union type is not allowed, as doing so would make the subclass property no longer assignable to the base class property's type.

In this example, `VagueGrade`'s `value` tries to add `| string` on top of the base class `NumericGrade`'s `number` type, causing a type error:

```
class NumericGrade {
    value = 0;
}

class VagueGrade extends NumericGrade {
    value = Math.random() > 0.5 ? 1 : "...";
    // Error: Property 'value' in type 'VagueGrade' is not
    // assignable to the same property in base type 'JustNumber'.
    //   Type 'string | number' is not assignable to type 'number'.
    //     Type 'string' is not assignable to type 'number'.
}

const instance: NumericGrade = new VagueGrade();

// Expected type: number
// Actual type: number | string
instance.value;
```

Abstract Classes

It can sometimes be useful to create a base class that doesn't itself declare the implementation of some methods, but instead expects a subclass to provide them. Marking a class as abstract is done by adding TypeScript's `abstract` keyword in front of the class name and in front of any method intended to be abstract. Those abstract method declarations skip providing a body in the abstract base class; instead, they are declared the same way an interface would be.

In this example, the `School` class and its `getStudentTypes` method are marked as `abstract`. Its subclasses—`Preschool` and `Absence`—are therefore expected to implement `getStudentTypes`:

```
abstract class School {
    readonly name: string;
```

```

    constructor(name: string) {
        this.name = name;
    }

    abstract getStudentTypes(): string[];
}

class Preschool extends School {
    getStudentTypes() {
        return ["preschooler"];
    }
}

class Absence extends School { }
// ~~~~~
// Error: Nonabstract class 'Absence' does not implement
// inherited abstract member 'getStudentTypes' from class 'School'.

```

An abstract class cannot be instantiated directly, as it doesn’t have definitions for some methods that its implementation may assume do exist. Only nonabstract (“concrete”) classes can be instantiated.

Continuing the `School` example, attempting to call `new School` would result in a TypeScript type error:

```

let school: School;

school = new Preschool("Sunnyside Daycare"); // Ok

school = new School("somewhere else");
// Error: Cannot create an instance of an abstract class.

```

Abstract classes are often used in frameworks where consumers are expected to fill out details of a class. The class may be used as a type annotation to indicate values must adhere to the class—as with the earlier example of `school: School`—but creating new instances must be done with subclasses.

Member Visibility

JavaScript includes the ability to start the name of a class member with `#` to mark it as a “private” class member. Private class members may only be accessed by instances of

that class. JavaScript runtimes enforce that privacy by throwing an error if an area of code outside the class tries to access the private method or property.

TypeScript's class support predates JavaScript's true `#` privacy, and while TypeScript supports private class members, it also allows a slightly more nuanced set of privacy definitions on class methods and properties that exist solely in the type system. TypeScript's member visibilities are achieved by adding one of the following keywords before the declaration name of a class member:

public (default)

Allowed to be accessed by anybody, anywhere

protected

Allowed to be accessed only by the class itself and its subclasses

private

Allowed to be accessed only by the class itself

These keywords exist purely within the type system. They're removed along with all other type system syntax when the code is compiled to JavaScript.

Here, `Base` declares two `public` members, one `protected`, one `private`, and one true private with `#truePrivate`. `Subclass` is allowed to access the `public` and `protected` members but not `private` or `#truePrivate`:

```
class Base {
  isPublicImplicit = 0;
  public isPublicExplicit = 1;
  protected isProtected = 2;
  private isPrivate = 3;
  #truePrivate = 4;
}

class Subclass extends Base {
  examples() {
    this.isPublicImplicit; // Ok
    this.isPublicExplicit; // Ok
    this.isProtected; // Ok

    this.isPrivate;
```



```

        // Error: Property 'isPrivate' is private
        // and only accessible within class 'Base'.

        this.#truePrivate;
        // Property '#truePrivate' is not accessible outside
        // class 'Base' because it has a private identifier.
    }
}

new Subclass().isPublicImplicit; // Ok
new Subclass().isPublicExplicit; // Ok

new Subclass().isProtected;
//
// Error: Property 'isProtected' is protected
// and only accessible within class 'Base' and its subclasses.

new Subclass().isPrivate;
//
// Error: Property 'isPrivate' is private
// and only accessible within class 'Base'.

```

The key difference between TypeScript's member visibilities and JavaScript's true private declarations is that TypeScript's exist only in the type system, while JavaScript's also exist at runtime. A TypeScript class member declared as `protected` or `private` will compile to the same JavaScript code as if they were declared `public` explicitly or implicitly. As with interfaces and type annotations, visibility keywords are erased when outputting JavaScript. Only `#` private fields are truly private in runtime JavaScript.

Visibility modifiers may be marked along with `readonly`. To declare a member both as `readonly` and with an explicit visibility, the visibility comes first.

This `TwoKeywords` class declares its `name` member as both `private` and `readonly`:

```

class TwoKeywords {
    private readonly name: string;

    constructor() {
        this.name = "Anne Sullivan"; // Ok
    }

    log() {

```



```

        console.log(this.name); // Ok
    }
}

const two = new TwoKeywords();

two.name = "Savitribai Phule";
// ~~~~
// Error: Property 'name' is private and
// only accessible within class 'TwoKeywords'.
// ~~~~
// Error: Cannot assign to 'name'
// because it is a read-only property.

```

Note that it is not permitted to mix TypeScript’s old member visibility keyword with JavaScript’s new `#` private fields. Private fields are always private by default, so there’s no need to additionally mark them with the `private` keyword.

Static Field Modifiers

JavaScript allows declaring members on a class itself—rather than its instances—using the `static` keyword. TypeScript supports using the `static` keyword on its own and/or with `readonly` and/or with one of the visibility keywords. When combined, the visibility keyword comes first, then `static`, then `readonly`.

This `Question` class puts them all together to make its `static` `prompt` and `answer` properties both `readonly` and `protected`:

```

class Question {
    protected static readonly answer: "bash";
    protected static readonly prompt =
        "What's an ogre's favorite programming language?";

    guess(getAnswer: (prompt: string) => string) {
        // Ok
        const answer = getAnswer(Question.prompt);

        // Ok
        if (answer === Question.answer) {
            console.log("You got it!");
        } else {
            console.log("Try again...")
        }
    }
}

```

```
    }  
}  
  
Question.answer;  
//      ~~~~~  
// Error: Property 'answer' is protected and only  
// accessible within class 'Question' and its subclasses.
```

Using read-only and/or visibility modifiers to static class fields is useful for restricting those fields from being accessed or modified outside their class.

This part introduced a plethora of type system features and syntaxes around classes:

- Declaring and using class methods and properties
- Marking properties `readonly` and/or optional
- Using class names as types in type annotations
- Implementing interfaces to enforce class instance shapes
- Extending classes, along with assignability and override rules for subclasses
- Marking classes and methods as abstract
- Adding type system modifiers to class fields